

Kompendium: Programowanie Obiektowe w Javie (Przygotowanie do Kolokwium)

Ten dokument został stworzony, aby przeprowadzić Cię od podstaw do zaawansowanych konceptów pojawiających się na kolokwium z programowania obiektowego. Pamiętaj, aby przed kolokwium pobrać ten plik, by mieć pod ręką gotowe struktury kodu (tzw. "gotowce") oraz wyjaśnienia. Opracowanie powstało na bazie analizy kolokwium z lat 2021, 2023 i 2024.

1. Absolutne Podstawy (Co musisz wiedzieć na start)

Klasa, Obiekt, Pola i Metody

W programowaniu obiektowym wszystko opiera się na klasach i obiektach. **Klasa** to przepis (np. przepis na ciasto), a **Obiekt** to fizyczna rzecz stworzona z tego przepisu (np. konkretne upieczone ciasto). **Pola** to cechy obiektu (np. smak, waga), a **Metody** to czynności, które obiekt może wykonać.

```
public class Car {  
    // 1. POLA (cechy)  
    private String brand;  
    private int speed;  
  
    // 2. KONSTRUKTOR (metoda wywoływana przy tworzeniu obiektu)  
    public Car(String brand) {  
        this.brand = brand; // 'this.brand' to pole klasy, 'brand' to argument z nawiasu  
        this.speed = 0;  
    }  
  
    // 3. METODY (czynności)  
    public void accelerate(int amount) {  
        this.speed += amount;  
    }  
  
    // 4. GETTER (akcesor - pozwala odczytać prywatne pole z zewnątrz)  
    public String getBrand() {  
        return this.brand;  
    }  
}
```

Modyfikatory dostępu

Na kolokwium często jest mowa o "prywatnych polach" i "publicznych metodach". Używamy ich, by kontrolować, co z zewnątrz może modyfikować nasze dane.

Modyfikator	Znaczenie	Kiedy używać na kolokwium?
private	Dostęp tylko wewnątrz danej klasy.	Zawsze używaj dla pól (zmiennych w klasie), chyba że polecenie mówi inaczej.
public	Dostęp z każdego miejsca w programie.	Używaj dla konstruktorów i większości metod .
protected	Dostęp w tej samej paczce i klasach dziedziczących.	Stosuj, gdy tworzysz hierarchię dziedziczenia i klasa podrzędna musi znać pola rodzica.

Słowa kluczowe: static i final

- **static**: Zmienna lub metoda należy do całej klasy, a nie do konkretnego obiektu. Wywołujesz ją tak: NazwaKlasy.metoda(), a nie na obiekcie. Na kolokwium często prosi się o "statyczną metodę fabrykującą/fromCsv" (np. kolokwium 2021, 2023).
- **final**: Po przypisaniu wartości, nie można jej już zmienić. Jeśli dotyczy klasy - nie można po niej dziedziczyć.

2. Analiza krok po kroku: Kolokwium 2024 (Zegary)

To kolokwium testuje wiedzę z tworzenia **klas abstrakcyjnych**, rzucania **wyjątków** oraz korzystania z biblioteki dat/czasu. Przerobimy zadanie powoli.

Krok 1: Klasa abstrakcyjna i wyrzucanie wyjątków

Zadanie: "Napisz klasę abstrakcyjną Clock przechowującą wewnętrznie stan zegara, posiadającą publiczne metody: setCurrentTime (ustawia czas na bieżący), setTime (przyjmuje godzinę, minutę, sekundę). Jeżeli dane nie są poprawne, należy rzucić wyjątek IllegalArgumentException z wiadomością. toString zwracającą format hh:mm:ss. Sposób przechowywania zaproponuj samodzielnie."

```
import java.time.LocalDateTime; // Wbudowana klasa Javy do trzymania samego czasu
```

```
// 1. "Napisz klasę abstrakcyjną Clock" - dodajemy słowo 'abstract'
```

```
public abstract class Clock {
```

```
    // 2. "przechowującą wewnętrznie stan zegara" - sposób wybieramy sami. Najłatwiej użyć  
    LocalDateTime.
```

```
    protected LocalDateTime time; // Używam protected, by klasy dziedziczące miały łatwy  
    dostęp
```

```
    // 3. Metoda setCurrentTime - pobiera aktualny czas z systemu
```

```
    public void setCurrentTime() {  
        this.time = LocalDateTime.now();  
    }
```

```
    // 4. Metoda setTime - przyjmuje h, m, s.
```

```
    public void setTime(int hour, int minute, int second) {  
        // Sprawdzanie poprawności i rzucanie wyjątku ("należy rzucić wyjątek  
        IllegalArgumentException")  
        if (hour < 0 || hour > 23) {  
            throw new IllegalArgumentException("Godzina " + hour + " nie mieści się w zakresie  
0-23.");  
        }  
        if (minute < 0 || minute > 59) {  
            throw new IllegalArgumentException("Minuta " + minute + " nie mieści się w  
zakresie 0-59.");  
        }  
        if (second < 0 || second > 59) {  
            throw new IllegalArgumentException("Sekunda " + second + " nie mieści się w  
zakresie 0-59.");  
        }  
    }
```

```
    // Jeśli wszystko jest OK, zapisujemy czas
```

```
    this.time = LocalDateTime.of(hour, minute, second);  
}
```

```
// 5. Nadpisanie metody toString ("zwraca napis zawierający godzinę w formacie  
hh:mm:ss")
```

```
@Override
```

```
public String toString() {  
    if (this.time == null) {  
        return "00:00:00"; // Zabezpieczenie, gdy czas nie został jeszcze ustawiony  
    }  
    // String.format() to świetny sposób na dodawanie zer wiodących (%02d znaczy 2  
    cyfry, uzupełnij zerami)  
    return String.format("%02d:%02d:%02d", time.getHour(), time.getMinute(),  
    time.getSecond());  
}
```

```
}  
}
```

Wyjaśnienie ważnych elementów z kodu powyżej:

- **abstract** oznacza, że nie możemy stworzyć obiektu klasy `Clock` (czyli zabronione jest `new Clock()`). Służy ona jako baza (szablon) dla innych, bardziej sprecyzowanych zegarów (np. Analogowy, Cyfrowy).
- **throw new IllegalArgumentException(...)** - w ten sposób "wybuchamy" błędem, gdy ktoś poda błędne dane, np. godzinę 25. Program się wtedy zatrzymuje, chyba że w innym miejscu "złapiemy" ten błąd (try-catch).
- **@Override** - znacznik (adnotacja), który mówi kompilatorowi: "teraz zmieniam domyślne działanie metody z klasy wyżej". `toString()` to metoda istniejąca w każdej klasie w Javie domyślnie, więc tu ją nadpisujemy pod własne potrzeby.

3. Analiza krok po kroku: Kolokwium 2023 (Mapa, Dziedziczenie)

Tutaj sprawdzano umiejętność budowania hierarchii, korzystania z list i typów wyliczeniowych (Enumów).

Krok 1 i 2: Klasy ze stałymi polami i użycie List

Zadanie: "Zdefiniuj klasę `Point` posiadającą dwa publiczne, ostateczne pola (`x`, `y`) ustawiane przez konstruktor. Zdefiniuj klasę `Polygon` posiadającą prywatną listę punktów ustawianą przez konstruktor."

```
import java.util.ArrayList;  
import java.util.List;  
  
// Krok 1: Punkt  
public class Point {  
    // "ostateczne pola" -> słowo kluczowe 'final'  
    // "liczby zmiennoprzecinkowe" -> double  
    public final double x;  
    public final double y;  
  
    // Konstruktor ustawiający te pola  
    public Point(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

```
}
```

```
// Krok 2: Polygon (Wielokąt)
public class Polygon {
    // "prywatna lista punktów"
    // List to tylko Interfejs (opis co można robić), a ArrayList to fizyczna implementacja
    private List<Point> points;

    // Konstruktor
    public Polygon(List<Point> points) {
        // Dobrą praktyką jest kopiowanie listy lub upewnienie się, że nie jest pusta,
        // ale na kolokwium wystarczy proste przypisanie:
        this.points = new ArrayList<>(points);
    }

    // (Możesz tu dodać getter do punktów na przyszłość)
    public List<Point> getPoints() {
        return this.points;
    }
}
```

Krok 5 i 6: Dziedziczenie (extends)

Zadanie: "Napisz klasę Land dziedziczącą po klasie Polygon, napisz taki sam konstruktor. Napisz klasę City dziedziczącą po Polygon z ostateczną zmienną center i polem nazwa."

```
// "dziedzicząca po" -> używamy słowa 'extends'
public class Land extends Polygon {

    // Taki sam konstruktor jak w rodzicu.
    // Musimy przekazać argumenty do rodzica za pomocą 'super()'
    public Land(List<Point> points) {
        super(points);
    }
}

// Kolejna klasa dziedzicząca
public class City extends Polygon {
    public final Point center;
    private String name;

    // Przykładowy konstruktor (opisany w kroku 6 w 2023)
    // Konstruktor generujący wierzchołki kwadratu pominięty dla czytelności dziedziczenia
    public City(String name, Point center, List<Point> wallPoints) {
        super(wallPoints); // Wywołanie konstruktora Polygon
    }
}
```

```
        this.name = name;
        this.center = center;
    }
}
```

Krok 10: Typy wyliczeniowe (Enum)

Zadanie: "W klasie Resource zdefiniuj publiczny typ wyliczeniowy Type {Coal, Wood, Fish}." Enumu używa się, gdy mamy z góry znaną, zamkniętą listę możliwości.

```
public class Resource {
    // 1. Zdefiniowanie typu wyliczeniowego wewnątrz klasy
    public enum Type {
        Coal, Wood, Fish
    }

    // Pola ostateczne
    public final Point position;
    public final Type type;

    public Resource(Point position, Type type) {
        this.position = position;
        this.type = type;
    }
}
```

4. Jak bezbłędnie czytać pliki (CSV, TXT) - MUST KNOW!

W prawie każdym kolokwium (np. z 2021: confirmed_cases.csv, deaths.csv; z 2024: strefy.csv) pojawia się parsowanie plików tekstowych. Należy wykuć na blachę odczyt za pomocą `java.nio.file.Files`.

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.List;
import java.io.IOException;

public class FileReaderUtils {

    // Przykład metody uniwersalnej
```

```

public static void readCsvFile(String filePath) {
    Path path = Paths.get(filePath); // Konwersja Stringa na obiekt ścieżki

    // Operacje wejścia/wyjścia mogą rzucić IOException, więc musimy użyć try-catch
    try {
        // Czyta od razu WSZYSTKIE linie z pliku i zapisuje je jako Listę Stringów
        List<String> lines = Files.readAllLines(path);

        for (int i = 0; i < lines.size(); i++) {
            String line = lines.get(i);

            // Często pierwszy wiersz to nagłówki, możemy go pominąć
            if (i == 0) {
                continue;
            }

            // "Rozdzielenie" linii według znaku, np. średnika (;) lub przecinka (,)
            String[] columns = line.split(";");

            // Teraz odwołujemy się do kolumn: columns[0], columns[1] itd.
            // Pamiętaj, że dane to wciąż napisy (String)!
            // Jeśli potrzebujesz liczby, musisz ją przekonwertować:
            // int number = Integer.parseInt(columns[1]);
            // double val = Double.parseDouble(columns[2]);
        }
    } catch (IOException e) {
        System.err.println("Błąd odczytu pliku: " + e.getMessage());
        // Zależnie od polecenia w zadaniu, można tu rzucić też np. RuntimeException
    }
}

```

5. Poradnik Przetwarzania na Kolokwium

1. **Czytaj od deski do deski na start:** Często kolejne kroki wyjaśniają, jak zbudować poprzednie. Nie wiesz jak coś zrobić w Kroku 2? Często Krok 4 narzuca rozwiązanie.
2. **Pisz zwięźle.** Jeśli czegoś nie wiesz – zostaw metody puste z napisanym `return null`; (lub rzucające wyjątek), żeby kod **się kompilował**. Często punkty dostaje się za samą strukturę i definicję nagłówka metody.
3. **Struktura danych ma znaczenie:** Używaj `List` dla kolekcji z zachowaniem kolejności, `Set` gdy nie może być duplikatów (np. 2023 zasoby), a `Map<Klucz, Wartość>` do szybkiego wyszukiwania.
4. **Polimorfizm w pigułce:** Jeśli masz klasę rodzica (`Pojazd`) i klasy pochodne (`Rower`,

Auto), możesz stworzyć listę rodziców: `List<Pojazd> lista = new ArrayList<>();` i dodać do niej zarówno Rower, jak i Auto! To często testowane na zaliczeniach (np. z 2021 krok 2: zwracanie polimorficznego obiektu typu Country).

Wklej te fragmenty do swojego projektu, upewnij się, że masz poinstalowane niezbędne biblioteki (jak Jackson dla XML-a, czy JUnit do testów, jeśli są wspomniane w konkretnym poleceniu) i modyfikuj je na bieżąco podczas kolokwium!

Zadanie 1 dla Ciebie (Spróbuj najpierw pomyśleć, jak byś to napisał)

Wyobraź sobie, że piszemy początek zadania na kolokwium. Twoje polecenie to:

"Napisz klasę `Book`. Klasa powinna posiadać dwa prywatne pola: napis `title` oraz ostateczną (final) liczbę całkowitą `pages`. Pola te powinny być ustawiane za pomocą publicznego konstruktora."

// 1. Definicja klasy

```
public class Book {
```

// 2. Pola klasy (cechy)

// "prywatne pole napis title"

```
private String title;
```

// "ostateczna liczba całkowita pages"

// Słowo 'final' oznacza, że po pierwszym przypisaniu nie będzie można już zmienić liczby stron.

```
private final int pages;
```

// 3. Publiczny konstruktor

// Konstruktor ZAWSZE nazywa się dokładnie tak samo jak klasa i nie ma typu zwracanego (nawet void).

```
public Book(String title, int pages) {
```

// Słowo 'this' oznacza "ten konkretny obiekt".

// this.title to pole klasy, a samo title to argument podany w nawiasie powyżej.

```
this.title = title;
```

```
this.pages = pages;
```

```
}
```

```
}
```

🔗 Zadanie 2 dla Ciebie

"Dodaj do konstruktora klasy `Book` sprawdzanie poprawności. Książka musi mieć więcej niż 0 stron. Jeżeli podana liczba stron jest mniejsza lub równa 0, rzuć wyjątek `IllegalArgumentException` z odpowiednią wiadomością."

```
public class Book {
    private String title;
    private final int pages;

    public Book(String title, int pages) {
        // Sprawdzanie poprawności ZAWSZE robimy przed przypisaniem wartości
        if (pages <= 0) {
            // throw new "rzuca" błędem, przerywając dalsze działanie programu.
            throw new IllegalArgumentException("Liczba stron musi być większa niż 0!");
        }

        this.title = title;
        this.pages = pages;
    }

    // Często na kolokwium proszą o akcesory (getter), by móc odczytać prywatne pole
    public String getTitle() {
        return this.title;
    }
}
```

Zadanie do samodzielnego napisania:

Napisz klasę `BankAccount` (Konto bankowe). Klasa powinna mieć dwa prywatne pola: `ownerName` (`String`) oraz `balance` (`double` - liczba zmiennoprzecinkowa). Napisz konstruktor, który przyjmuje te dwie wartości i je ustawia. Zabezpiecz konstruktor: jeśli początkowy `balance` (stan konta) jest mniejszy od zera, rzuć wyjątek `IllegalArgumentException` z wiadomością "Stan konta nie może być ujemny".

```
public class BankAccount {
    private String ownerName;
    private double balance;

    // Zmienione "string" na "String"
    public BankAccount(double balance, String ownerName) {

        // Zmienione <= na <
        if (balance < 0) {
            // Zmienione throw. na throw new + poprawiona literówka w Exception
            throw new IllegalArgumentException("Stan konta nie może być ujemny");
        }

        this.balance = balance;
        this.ownerName = ownerName;
    }
}
```

Zadanie do samodzielnego napisania (2 klasy):

1. Napisz klasę `Document` . Niech ma jedno prywatne pole `title` (typu `String`) i publiczny konstruktor, który je ustawia.
2. Napisz klasę `PdfDocument` , która dziedziczy po klasie `Document` (pamiętaj o `extends`).
3. W klasie `PdfDocument` dodaj prywatne pole `fileSize` (typu `int`).
4. Napisz konstruktor dla `PdfDocument` , który przyjmuje **dwie** rzeczy: `String title` oraz `int fileSize` . Wewnątrz niego wywołaj konstruktor rodzica (używając `super`), a następnie ustaw pole `fileSize` .

```
public class Document {  
    private String title;  
  
    public Document(String title) {  
        this.title = title;  
    }  
}
```

```
public class PdfDocument extends Document {  
    private int fileSize;  
  
    public PdfDocument(String title, int fileSize) {  
        super(title);  
        this.fileSize = fileSize;  
    }  
}
```

Twoja kolej na ćwiczenie! 🚀

Połączmy naszą klasę `Document` z nową wiedzą o listach.

Zadanie do samodzielnego napisania:

1. Napisz klasę `Folder`.
2. Dodaj w niej prywatne pole `documents`, które będzie listą przechowującą obiekty typu `Document` (z naszego poprzedniego zadania).
3. Napisz bezargumentowy konstruktor (czyli taki z pustymi nawiasami `public Folder() { ... }`), który wewnątrz przypisze do pola `documents` nową, pustą `ArrayList`.
4. Napisz publiczną metodę `addDocument(Document doc)`, która za pomocą metody `.add()` dorzuci przekazany dokument do naszej listy wewnątrz klasy.

Spróbuj napisać tę klasę i podesłaj tutaj!

Document.java

```
public class Document {
    private String title;

    public Document(String title) {
        this.title = title;
    }

    // Dodajmy getter, żeby móc wyświetlić tytuł później
    public String getTitle() {
        return title;
    }
}
```

Folder.java

```
import java.util.ArrayList;
import java.util.List;

public class Folder {
    // Pole przechowujące listę dokumentów
```

```

private List<Document> documents;

// Konstruktor bezargumentowy - inicjalizuje pustą listę
public Folder() {
    this.documents = new ArrayList<>();
}

// Metoda do dodawania dokumentu do listy
public void addDocument(Document doc) {
    this.documents.add(doc);
}

// Dodatkowa metoda, żebyś mógł sprawdzić zawartość folderu
public void showAllDocuments() {
    System.out.println("Zawartość folderu:");
    for (Document doc : documents) {
        System.out.println("- " + doc.getTitle());
    }
}
}

```

Main.java

```

public class Main {
    public static void main(String[] args) {
        // 1. Tworzymy folder
        Folder myFolder = new Folder();

        // 2. Tworzymy kilka dokumentów
        Document doc1 = new Document("Notatki z Javy");
        Document doc2 = new Document("Lista Zakupów");

        // 3. Dodajemy dokumenty do folderu
        myFolder.addDocument(doc1);
        myFolder.addDocument(doc2);

        // 4. Wyświetlamy, co mamy w folderze
        myFolder.showAllDocuments();
    }
}

```

KLASY ABSTRAKCYJNE:

// 1. Dodajemy słowo 'abstract' przed 'class'

```
public abstract class Animal {  
    private String name;
```

```
  
    public Animal(String name) {  
        this.name = name;  
    }  
  

```

// Zwykła metoda - ma ciało i działa normalnie

```
public String getName() {  
    return name;  
}  
  

```

// 2. METODA ABSTRAKCYJNA - zauważ brak { } i średnik na końcu!

// Zmuszamy każde "dziecko" do określenia, jaki wydaje dźwięk.

```
public abstract void makeSound();  
}
```

// -----

// Klasa Dziecko

```
public class Dog extends Animal {
```

```
  
    public Dog(String name) {  
        super(name);  
    }  
  

```

// 3. Klasa dziedzicząca MUSI nadpisać (@Override) metodę abstrakcyjną

// Inaczej kod w ogóle się nie skompiluje!

@Override

```
public void makeSound() {  
    System.out.println("Hau hau!");  
}  
}
```

Zadanie do samodzielnego napisania (2 klasy):

1. Napisz klasę abstrakcyjną `Product` .

- Dodaj prywatne pole `name` (typu `String`) i publiczny konstruktor, który je ustawia.
- Dodaj publiczną, **abstrakcyjną** metodę o nazwie `getPrice` , która ma przyjmować dwa argumenty typu `int` : `year` oraz `month` . Metoda ma zwracać wartość typu `double` . Pamiętaj: jako że jest abstrakcyjna, nie ma klamerek `{ }` , tylko średnik na końcu!

2. Napisz klasę `NonFoodProduct` , która dziedziczy po `Product` .

- Napisz dla niej konstruktor przyjmujący `String name` i wywołujący `super(name)` .
- Zaimplementuj (nadpisz) metodę abstrakcyjną `getPrice` . Na potrzeby naszego testu niech po prostu zwraca jakąś stałą liczbę, np. `return 99.99;` . (Ważne, by zgadzał się typ zwracany i argumenty).

```
public abstract class Product {  
    private String name;  
  
    public Product(String name) {  
        this.name = name;  
    }  
  
    public abstract double getPrice(int year, int month);  
}
```

```
public class NonFoodProduct extends Product {  
  
    // 1. Poprawiona literówka (Non zamiast Nom)  
    public NonFoodProduct(String name) {  
        super(name);  
    }  
  
    // 2. Wypełnienie "kontraktu" - nadpisanie metody z klasy Product
```

```

@Override
public double getPrice(int year, int month) {
    return 99.99; // Zwracamy przykładową wartość double
}
}

```

Fragment 6: Enums (Typy wyliczeniowe) i `toString()`

Krótką teoria:

1. **Enum:** Używamy go, gdy mamy z góry znaną, zamkniętą listę możliwości (np. dni tygodnia, pory roku, stany zamówienia). Zamiast używać zwykłego tekstu (typu `String`), w którym łatwo o literówkę, tworzymy własny typ.

2. **`toString()`:** Każdy obiekt w Javie można wydrukować na ekranie

(`System.out.println(mojObiekt)`). Domyślnie jednak Java wydrukuje jakieś "krzaczkę"

(np. `Task@7a81197d` - adres w pamięci). Aby to naprawić, **nadpisujemy** (`@Override`)

metodę `toString()`, mówiąc Javie, jak ładnie opisać nasz obiekt tekstem.

```

public class Pizza {
    // 1. Definiujemy Enum (zazwyczaj piszemy opcje WIELKIMI LITERAMI)
    public enum Size {
        SMALL, MEDIUM, LARGE
    }

    private String name;
    private Size size; // Używamy naszego Enuma jako typu zmiennej!

    public Pizza(String name, Size size) {
        this.name = name;
        this.size = size;
    }

    // 2. Nadpisujemy metodę toString, żeby ładnie opisać pizzę
    @Override
    public String toString() {

```

```
        return "Pizza: " + this.name + ", rozmiar: " + this.size;
    }
}
```

Zadanie do samodzielnego napisania:

1. Napisz klasę `ProjectTask` .
2. Wewnątrz tej klasy zdefiniuj publiczny `enum` o nazwie `State` (Stan), który będzie miał trzy opcje: `TODO` , `DOING` , `DONE` .
3. Dodaj dwa prywatne pola do klasy: `String title` (tytuł zadania) oraz `State state` (stan zadania).
4. Napisz publiczny konstruktor, który przyjmuje te dwa parametry i je ustawia.
5. Nadpisz (`@Override`) metodę `public String toString()` , która zwróci napis w takim formacie:
`"Zadanie: [title] jest obecnie w stanie: [state]"` .

```
public class ProjectTask {
    public enum State{
        TODO, DOING, DONE
    }

    private String title;
    private State state;

    public ProjectTask(State state, String title) {
        this.state = state;
        this.title = title;
    }

    @Override
    public String toString() {
        return "Zadanie: " + this.title + ", jest obecnie w stanie: "
+ this.state;
    }
}
```

```
}
```

Fragment 7: Metody statyczne (`static`) i dzielenie tekstu (`String.split`)

Na kolokwiach prowadzący zawsze dają jakiś plik (np. `.csv` lub `.txt`), z którego trzeba wyciągnąć dane i zamienić je na obiekty Javy. Wymaga to tzw. "metody wytwórczej" (często każą ją nazwać `fromCsv` lub `parseLine`).

Krótką teoria:

1. **`static` (Statyczny):** Zwykle metody (jak Twoje `toString()`) wywołujemy na konkretnym, gotowym obiekcie (np. `mojeZadanie.toString()`). Metoda `static` należy do całej klasy, a nie do jednego obiektu. Wywołujemy ją po nazwie klasy: `Math.max(2, 5)` albo `ProjectTask.jakasMetoda()`. Używamy ich często do tworzenia nowych obiektów z jakiegoś tekstu.
2. `String.split("znak")`: Ta metoda tnie długi tekst na kawałki i wrzuca je do tablicy `[]`.
3. **Zamiana tekstu na liczbę:** Tekst `"25"` to dla Javy nie to samo co liczba `25`. Żeby zamienić tekst na liczbę całkowitą, musimy użyć `Integer.parseInt("25")`.

```
public class Car {
    private String brand;
    private int speed;

    public Car(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }

    // METODA STATYCZNA tworząca samochód z linijki tekstu
    // Zauważ, że zwraca obiekt typu 'Car'
    public static Car parseLine(String line) {
        // Zakładamy, że line wygląda tak: "BMW;120"

        // 1. Tniemy tekst według średnika
```

```

String[] parts = line.split(";");

// parts[0] to teraz "BMW"
// parts[1] to teraz "120"

// 2. Wyciągamy dane
String brand = parts[0];
// 3. Zamieniamy tekst na liczbę
int speed = Integer.parseInt(parts[1]);

// 4. Tworzymy i zwracamy nowy obiekt
return new Car(brand, speed);
}
}

```

Zadanie do samodzielnego napisania:

1. Napisz klasę `Student`.
2. Dodaj dwa prywatne pola: `String name` oraz `double grade` (ocena).
3. Napisz zwykły, publiczny konstruktor, który przyjmuje i ustawia te dwa pola.
4. Napisz **publiczną, statyczną metodę** o nazwie `fromCsv`, która:
 - Przyjmie jako argument napis (linijkę z pliku): `String line`.
 - Zwróci nowy obiekt typu `Student`.
 - Załóż, że linijka będzie wyglądała tak: `"Jan Kowalski,4.5"`. (Zwróć uwagę, że są oddzielone **przecinkiem!**).
 - Wewnątrz metody podziel tekst, zamień odpowiedni kawałek na ułamek używając `Double.parseDouble(...)` i zwróć za pomocą słowa `return` nowy obiekt studenta.

```

public class Student {
    private String name;

```

```
private double grade;

// Konstruktor - super, że pamiętałeś o poprawnej kolejności
argumentów!
public Student(double grade, String name) {
    this.grade = grade;
    this.name = name;
}

// Zwraca Typ (Student), nazwa to fromCsv
public static Student fromCsv(String line) {

    // 1. Tniemy tekst według przecinka i zapisujemy do tablicy
    'parts'
    String[] parts = line.split(",");

    // parts[0] to u nas imię, np. "Jan Kowalski"
    // parts[1] to u nas ocena jako tekst, np. "4.5"

    // 2. Wyciągamy dane z tablicy
    String parsedName = parts[0];

    // 3. Zamieniamy tekst na liczbę zmiennoprzecinkową używając
    klasy Double
    double parsedGrade = Double.parseDouble(parts[1]);

    // 4. Tworzymy i zwracamy nowy obiekt klasy Student
    // (Zwróć uwagę, że w Twoim konstruktorze najpierw jest ocena,
    a potem imię)
    return new Student(parsedGrade, parsedName);
}
}
```

Fragment 8: Czytanie z pliku (`Files.readAllLines`) i Pętla

Krótką teoria:

Żeby wczytać plik w Javie, potrzebujemy zaledwie kilku linijek, ale musimy pamiętać o tzw. **wyjątkach kontrolowanych** (takich jak `IOException`). Jeśli Java próbuje otworzyć plik, który nie istnieje, musi wiedzieć, co zrobić w razie wybuchu. Dlatego kod czytający z pliku zamykamy w bloku `try-catch` (spróbuj to zrobić, a jak się nie uda, to złap błąd).

Druga sprawa to **pętla for-each**. Kiedy wyciągniemy już wszystkie linijki z pliku do listy, musimy przez nie przejść jedna po drugiej:

```
// Tłumaczenie: "Dla każdego napisu (nazywamy go 'line') znajdującego się w liście 'lines' zrób to co w klamerkach"
for (String line : lines) {
    // tutaj coś robimy z pojedynczą linijką
}
```

Zadanie do samodzielnego napisania:

Twoim zadaniem jest **uzupełnienie środka** tego kodu (tam gdzie są komentarze `// TODO`).

1. Skopiuj poniższy szkielet.
2. W miejscu `TODO 1` napisz pętlę `for` , która przejdzie przez wszystkie linijki w liście `lines` .
3. W miejscu `TODO 2` (wewnątrz pętli) wywołaj metodę `Student.fromCsv(line)` by stworzyć obiekt studenta, i zapisz go do zmiennej.
4. W miejscu `TODO 3` dodaj tego studenta do naszej głównej listy `studentsList` używając metody `.add()` .

```
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.List;
import java.io.IOException;
```

```

public class Database {

    public List<Student> loadStudents(String pathToFile) {

        // 1. Przygotowujemy pusty "koszyk" na naszych studentów
        List<Student> studentsList = new ArrayList<>();

        try {
            // 2. Czytamy wszystkie linijki z pliku do listy tekstów
            (Stringów)
            List<String> lines =
Files.readAllLines(Paths.get(pathToFile));

            // ROZWIĄZANIE:

            // TODO 1: Pętla for-each
            // Tłumaczenie: "Dla każdego napisu (nazywamy go 'line')
wyciągniętego z listy 'lines' zrób to:"
            for (String line : lines) {

                // TODO 2: Tworzymy studenta z jednej linijki
                // Używamy magicznej metody fromCsv, którą napisałeś
wcześniej!
                Student newStudent = Student.fromCsv(line);

                // TODO 3: Wrzucamy gotowego studenta do naszego
"koszyka"
                studentsList.add(newStudent);
            }

        } catch (IOException e) {
            // Ten kod wykona się TYLKO wtedy, gdy np. plik nie
istnieje albo podaliśmy złą ścieżkę
            System.out.println("Uwaga! Nie udało się otworzyć pliku: "
+ pathToFile);
        }

        // Zwracamy pełny, załadowany koszyk
    }
}

```

```
        return studentsList;
    }
}
```

Fragment 9: Sortowanie Listy obiektów (`Comparator`)

Krótką teoria:

Kiedy masz listę zwykłych liczb (np. `5, 1, 8`), Java wie, jak je posortować od najmniejszej do największej. Ale kiedy masz listę obiektów (np. `Student`), Java "głupieje" – nie wie, czy ma sortować tych studentów alfabetycznie po imieniu, czy rosnąco po ocenach. Musimy jej to powiedzieć!

Służy do tego tzw. **Komparator** (czyli z ang. *porównywacz*). Najnowocześniejszym i najkrótszym sposobem w Javie jest użycie metody `.sort()` na naszej liście i przekazanie jej narzędzia `Comparator.comparing(...)`.

```
import java.util.List;
import java.util.Comparator; // Musimy to zaimportować!

public class Garage {

    public void sortCarsBySpeed(List<Car> cars) {

        // Magiczna linijka sortująca:
        // Tłumaczenie: "Weź listę 'cars', posortuj ją, porównując (comparing) prędkości (getSpeed)
każdego z aut"
        cars.sort(Comparator.comparing(Car::getSpeed));

        // Zauważ ten dziwny zapis: Car::getSpeed
        // To tzw. referencja do metody. Mówi Javie: "Użyj tej konkretnej metody, by wyciągnąć
wartość do sortowania"
    }
}
```

Zadanie do samodzielnego napisania:

1. Wróć do swojej klasy `Student`. Zauważ, że pole `grade` (ocena) jest `private`. Żeby komparator miał do niego dostęp, dodaj w klasie `Student` publiczną metodę (getter):

Java



```
public double getGrade() {  
    return this.grade;  
}
```

2. Przejdź do klasy `Database`. Pamiętaj o dodaniu na samej górze pliku: `import java.util.Comparator;`.
3. Napisz w klasie `Database` nową, publiczną metodę o nazwie `sortStudents`.
4. Metoda powinna przyjmować jako argument listę studentów: `List<Student> students`.
5. Wewnątrz metody wywołaj na tej liście `.sort(...)`, używając komparatora, tak by posortował studentów po ocenach (wykorzystując `Student::getGrade`). Nie musi nic zwracać (typ `void`), ponieważ `.sort` po prostu zmienia kolejność wewnątrz istniejącej listy.

```
import java.nio.file.Files;  
import java.nio.file.Paths;  
import java.util.ArrayList;  
import java.util.Comparator; // Potrzebne do sortowania  
import java.util.List; metoda  
import java.io.IOException; // Potrzebne do obsługi błędów plików  
  
public class Database {  
  
    /**  
     * Metoda wczytująca studentów z pliku CSV.  
     * Realizuje wymagania dotyczące parsowania plików z treści  
kolokwiiów.  
     */  
}
```

```

public List<Student> loadStudents(String pathToFile) {
    List<Student> studentsList = new ArrayList<>();

    try {
        // Wczytujemy wszystkie linie [cite: 223]
        List<String> lines =
Files.readAllLines(Paths.get(pathToFile));

        for (String line : lines) {
            // Pomijamy puste linie, jeśli się zdarza
            if (line.trim().isEmpty()) continue;

            // Tworzymy obiekt za pomocą Twojej metody fromCsv
            Student s = Student.fromCsv(line);
            studentsList.add(s);
        }
    } catch (IOException e) {
        // Obsługa błędu, jeśli plik nie istnieje [cite: 295]
        System.err.println("Błąd: Nie można odczytać pliku " +
pathToFile);
    }

    return studentsList;
}

/**
 * WARIANT 1: Sortowanie rosnąco po ocenie (od 2.0 do 5.0)
 */
public void sortStudentsByGradeAsc(List<Student> students) {
    students.sort(Comparator.comparing(Student::getGrade));
}

/**
 * WARIANT 2: Sortowanie MALEJĄCO po ocenie (od 5.0 do 2.0)
 * To najczęstszy wymóg na kolokwiah (np. sortowanie po liczbie
zgonów lub różnicy czasu).
 */
public void sortStudentsByGradeDesc(List<Student> students) {

```

```

students.sort(Comparator.comparing(Student::getGrade).reversed());
}

/**
 * WARIANT 3: Sortowanie alfabetyczne po imieniu
 */
public void sortStudentsByName(List<Student> students) {
    students.sort(Comparator.comparing(Student::getName));
}
}

```

Fragment 10: Filtrowanie Listy (Instrukcja warunkowa w pętli)

Krótką teoria:

Filtrowanie polega na przejrzaniu całej listy i wybraniu z niej tylko tych elementów, które spełniają określony warunek.

Złota zasada dla początkujących: **nigdy nie usuwaj elementów ze starej listy podczas przechodzenia przez nią pętlą!** To powoduje błędy (tzw.

`ConcurrentModificationException`). Zamiast tego, tworzymy **nową, pustą listę** (nasz "nowy koszyk") i wrzucamy do niej tylko te elementy, które przejdą test.

```

import java.util.ArrayList;
import java.util.List;

public class Garage {

    // Metoda zwraca nową listę z wybranymi autami
    public List<Car> getFastCars(List<Car> allCars) {

        // 1. Tworzymy nowy, pusty koszyk na wyniki
        List<Car> fastCars = new ArrayList<>();

        // 2. Przechodzimy przez wszystkie auta ze starej listy
        for (Car car : allCars) {

```

```

// 3. Sprawdzamy warunek
if (car.getSpeed() > 200) {
    // 4. Jeśli warunek jest spełniony, dodajemy auto do nowego koszyka
    fastCars.add(car);
}
}

// 5. Zwracamy koszyk z samymi szybkimi autami
return fastCars;
}
}

```

Zadanie do samodzielnego napisania:

1. W klasie `Database` napisz publiczną metodę o nazwie `getPassedStudents`.
2. Metoda powinna **zwracać** `List<Student>` (listę studentów).
3. Metoda powinna **przyjmować** jako argument całą listę studentów: `List<Student> allStudents`.
4. Wewnątrz metody stwórz nową listę (np. o nazwie `passed`).
5. Użyj pętli `for-each`, żeby przejść przez `allStudents`.
6. Wewnątrz pętli użyj instrukcji `if` i metody `getGrade()`, żeby sprawdzić, czy ocena studenta jest **większa lub równa** (`>=`) `3.0`.
7. Jeśli tak, dodaj go do nowej listy.
8. Na samym końcu metody (poza pętlą!) zwróć nową listę używając `return`.

```

// Metoda zwraca Listę Studentów i przyjmuje Listę wszystkich
// Studentów
public List<Student> getPassedStudents(List<Student> allStudents) {

    // 1. Tworzymy nowy, pusty "koszyk" na wyniki
    List<Student> passed = new ArrayList<>();
}

```

```
// 2. Przechodzimy przez każdego studenta na liście bazowej
for (Student student : allStudents) {

    // 3. Sprawdzamy warunek zdania egzaminu (ocena większa lub
    równa 3.0)
    if (student.getGrade() >= 3.0) {

        // 4. Jeśli student spełnia warunek, dorzucamy go do
        nowego koszyka
        passed.add(student);
    }
}

// 5. Zwracamy koszyk tylko z tymi osobami, które zdały (ci z
ocena 2.0 tu nie trafili)
return passed;
}
```

Fragment 11: Mapy (Słowniki `Map<Klucz, Wartość>`)

Krótką teoria:

Zwykła `List` to jak kolejka w sklepie – elementy są ułożone jeden za drugim (indeksy 0, 1, 2...).

`Map` to z kolei **książka telefoniczna albo słownik**. Składa się z par: **Klucz** (np. nazwisko) i **Wartość** (np. numer telefonu).

Dzięki temu, jeśli chcesz znaleźć konkretnego studenta, nie musisz przeszukiwać całej listy pętlą. Pytasz mapę: "Daj mi studenta o kluczu 'Jan Kowalski'" i dostajesz go natychmiast.

Zasady gry:

1. **Unikalne klucze:** W mapie klucze nie mogą się powtarzać. Jeśli dodasz do mapy drugą osobę pod kluczem "Jan Kowalski"  stara wartość zostanie nadpisana nową.
2. **Definicja:** Przy tworzeniu podajemy dwa typy w nawiasach ostrych. Np. `Map<String, Student>` oznacza, że kluczem jest tekst (`String`), a przypisaną do niego wartością jest obiekt `Student`.
3. **Metody:** * Zamiast `.add()`, w mapach używamy `.put(klucz, wartość)` (włóż).
 - Do wyciągania danych używamy `.get(klucz)` (pobierz).

```
import java.util.HashMap;  
import java.util.Map;
```

```
public class University {
```

```
    public void demonstrateMap() {  
        // 1. Tworzenie pustej mapy (zwróć uwagę na HashMap po stronie 'new')  
        Map<String, Student> studentsMap = new HashMap<>();  
  
        // Tworzymy studentów  
        Student s1 = new Student(4.5, "Anna Nowak");  
        Student s2 = new Student(3.0, "Jan Kowalski");  
  
        // 2. Dodawanie do mapy (Kluczem jest imię, Wartością jest sam obiekt studenta)  
        studentsMap.put(s1.getName(), s1); // klucz: "Anna Nowak", wartość: obiekt s1
```

```
studentsMap.put(s2.getName(), s2); // klucz: "Jan Kowalski", wartość: obiekt s2
```

```
// 3. Wyciąganie z mapy BEZ UŻYCIA PĘTLI!
```

```
// Podajemy sam klucz i od razu dostajemy obiekt:
```

```
Student szukany = studentsMap.get("Anna Nowak");
```

```
System.out.println("Znalazłem: " + szukany.getGrade());
```

```
}
```

```
}
```

Zadanie do samodzielnego napisania:

1. W klasie `Database` napisz publiczną metodę o nazwie `createStudentMap`.
2. Metoda powinna **zwracać** mapę: `Map<String, Student>`.
3. Metoda powinna **przyjmować** listę: `List<Student> students`.
4. Wewnątrz metody stwórz nową, pustą `HashMap`.
5. Użyj pętli `for-each`, aby przejść przez każdego studenta w podanej liście.
6. Wewnątrz pętli użyj metody `.put(...)`, aby włożyć studenta do mapy. Jako klucz podaj imię studenta (użyj metody `getName()`), a jako wartość samego studenta.
7. Na końcu metody zwróć stworzoną mapę używając `return`.

```
public Map<String, Student> createStudentMap(List<Student> students)
{
    Map<String, Student> studentMap = new HashMap<>();

    for (Student student : students) {
        studentMap.put(student.getName(), student);
    }
    return studentMap;
}
```

Fragment 12: Grafika w Javie, czyli generowanie plików SVG

Krótką teorią:

Na kolokwium nie będziesz używać żadnych skomplikowanych bibliotek graficznych. Używa się formatu **SVG**.

Dlaczego? Bo plik `.svg` to tak naprawdę zwykły plik tekstowy, w którym znajdują się "tagi" (znaczniki) mówiące komputerowi, co ma narysować.

Zamiast "rysować pędzlem", piszemy tekst.

- Żeby narysować koło: `<circle cx="50" cy="50" r="40" fill="red" />` (gdzie `cx` i `cy` to środek, `r` to promień, a `fill` to kolor).
- Żeby narysować linię: `<line x1="0" y1="0" x2="100" y2="100" stroke="black" />`.

Zasady gry:

1. Każdy element rysowalny (np. Zegar, Wskazówka, Miasto) powinien mieć metodę `public String toSvg()`, która zwraca jego znacznik graficzny jako tekst. `+3`
2. Uważaj na cudzysłowy w tekście! W Javie cudzysłów `"` oznacza początek/koniec napisu. Jeśli chcesz wstawić cudzysłów do środka napisu, musisz go "uciec" używając ukośnika: `\`.
Np. `String tekst = "<circle r=\"5\" />";`
3. Gotowy tekst po prostu zapisujemy do pliku tekstowego z rozszerzeniem `.svg`.

```
import java.nio.file.Files;
import java.nio.file.Paths;
import java.io.IOException;
```

```
public class GraphicPoint {
    private int x;
    private int y;

    public GraphicPoint(int x, int y) {
        this.x = x;
```

```

        this.y = y;
    }

    // 1. Metoda zwracająca tekstowy znacznik SVG
    public String toSvg() {
        // Używamy ucieczki znaków (\") żeby wstawić cudzysłowy do zmiennych w SVG
        return "<circle cx=\"" + this.x + "\" cy=\"" + this.y + "\" r=\"10\" fill=\"blue\" />";
    }

    // 2. Zapisywanie do pliku
    public void saveToFile(String filename) {
        // Żeby plik zadziałał, musimy obudować nasz element głównymi tagami <svg>
        String fullSvg = "<svg height=\"200\" width=\"200\">\n" +
            this.toSvg() +
            "\n</svg>";

        try {
            // Zapisujemy nasz długi napis do pliku
            Files.writeString(Paths.get(filename), fullSvg);
            System.out.println("Zapisano grafikę!");
        } catch (IOException e) {
            System.out.println("Błąd zapisu pliku.");
        }
    }
}

```

Twoja kolej na ćwiczenie! 🚀

Zróbmy wycinek z kolokwium 2024 (Krok 9: Wskazówka zegara rysowana jako odcinek).

Rysowanie linii to podstawa. 📄 +2

Zadanie do samodzielnego napisania:

1. Napisz klasę `ClockHand` (Wskazówka).
2. Dodaj dwa prywatne pola całkowite (`int`): `endX` oraz `endY` (punkt końcowy wskazówki, zakładamy, że początek zawsze jest w punkcie 0,0).
3. Napisz publiczny konstruktor, który ustawi te wartości.
4. Napisz metodę `public String toSvg()`.
5. Metoda ma zwracać napis reprezentujący linię w SVG. Znacznik linii wygląda tak:

```
<line x1="0" y1="0" x2="WSTAW_TUTAJ_ENDX" y2="WSTAW_TUTAJ_ENDY"
stroke="black" />
```

Pamiętaj, że musisz posklejać tekst (`+`) z Twoimi zmiennymi (`this.endX` , `this.endY`) i nie zapomnieć o "uciekaniu" cudzysłowów (`\`) dla czarnego koloru!

```
public class ClockHand {
    private int endx;
    private int endy;

    public ClockHand(int endx, int endy) {
        this.endx = endx;
        this.endy = endy;
    }

    public String toSvg() {
        // Zwróć uwagę na sklejanie plusami!
        return "<line x1=\"0\" y1=\"0\" x2=\"" + this.endx + "\"
y2=\"" + this.endy + "\" stroke=\"black\" />";
    }
}
```

Fragment 13: Tworzenie własnych wyjątków

Krótką teoria:

Rzucanie wyjątków (jak `throw new IllegalArgumentException(...)`) już umiesz. Ale co, jeśli Java nie ma błędu o nazwie, która pasuje do Twojego problemu? Wtedy piszemy własny!

Aby stworzyć własny wyjątek, musimy po prostu napisać nową klasę, która **dziedziczy** (`extends`) po jednej z wbudowanych klas błędów w Javie. Najczęściej są to:

1. `Exception` – tzw. wyjątek kontrolowany. Jeśli metoda go rzuca, kod **nie skompiluje się**, dopóki ktoś go nie "złapie" w bloku `try-catch` (o to prosili w 2021 r.). 
2. `RuntimeException` – tzw. wyjątek niekontrolowany. Program po prostu wybuchą i się zamyka.

Zasady gry:

1. Tworzysz nową klasę z przyrostkiem `Exception` w nazwie.
2. Dopisz `extends Exception` lub `extends RuntimeException`.
3. Tworzysz konstruktor, który przyjmuje jakąś wiadomość tekstową (np. co poszło nie tak) i **przekazuje ją do rodzica** za pomocą `super(wiadomosc)`.

```
// 1. Dziedziczymy po RuntimeException
public class TooYoungException extends RuntimeException {

    // 2. Konstruktor przyjmuje wiek, który spowodował błąd
    public TooYoungException(int age) {

        // 3. Wywołujemy konstruktor rodzica (super), żeby sformatował
        ładną wiadomość
        // Metoda getMessage() na tym wyjątku zwróci właśnie ten
        sklejonny tekst.
        super("Osoba jest za młoda! Ma dopiero " + age + " lat.");
    }
}
```

```

if (age < 18) {
    throw new TooYoungException(age);
}

```

Klasa +metoda publiczna boolean

Krok 2.

Zdefiniuj klasę Polygon, która będzie posiadała prywatną listę punktów. Lista punktów powinna być ustawiana przez konstruktor. Wielokąt zbudowany jest z krawędzi łączących kolejne punkty w liście oraz ostatni z pierwszym punktem z listy.

W dalszej części zadania, obiekty klasy Polygon będą skrótowo nazywane wielokątami.

Krok 3.

W klasie Polygon zdefiniuj publiczną metodę inside(Point), która zwraca wartość logiczną określającą, czy dany argumentem metody punkt znajduje się w wielokącie, na rzecz którego wywoływana jest metoda. Zaimplementuj w niej algorytm podany pseudokodem:

Dane są: wielokąt *poly* i sprawdzany punkt *point*.

counter = 0

Dla każdej pary (*pa*, *pb*) punktów tworzących krawędź wielokąta *poly*:

Jeżeli *pa.y* > *pb.y*:

Zamień *pa* z *pb*

Jeżeli *pa.y* < *point.y* < *pb.y*:

d = *pb.x* - *pa.x*

Jeżeli *d* == 0:

x = *pa.x*

W przeciwnym razie:

a = (*pb.y* - *pa.y*) / *d*

b = *pa.y* - *a* * *pa.x*

x = (*point.y* - *b*) / *a*

Jeżeli *x* < *point.x*:

counter++

Zwróć prawdę, jeżeli *counter* jest nieparzysty, a fałsz w przeciwnym przypadku.

```

import java.util.ArrayList;
import java.util.List;

public class Polygon {
    private List<Point> points;

    public Polygon(List<Point> points) {
        this.points = new ArrayList<>(points);
    }
}

```

```

public boolean inside(Point point) {
    int counter = 0;
    int n = points.size();

    // Pętla przechodzi przez każdą parę punktów tworzących
    krawędź.
    // Używamy modulo (%), aby połączyć ostatni punkt na liście z
    pierwszym.
    for (int i = 0; i < n; i++) {
        Point pa = points.get(i);
        Point pb = points.get((i + 1) % n);

        // Zabezpieczenie przed modyfikacją oryginalnych punktów w
        liście

        // Tworzymy lokalne zmienne do ewentualnej zamiany
        double paX = pa.x;
        double paY = pa.y;
        double pbX = pb.x;
        double pbY = pb.y;

        // Jeżeli pa.y > pb.y: Zamień pa z pb
        if (paY > pbY) {
            double tempX = paX;
            double tempY = paY;

            paX = pbX;
            paY = pbY;

            pbX = tempX;
            pbY = tempY;
        }

        // Jeżeli pa.y < point.y < pb.y
        if (paY < point.y && point.y < pbY) {
            double d = pbX - paX;
            double x;

            if (d == 0) {

```

```

        x = paX;
    } else {
        // Wymuszamy operacje na zmiennoprzecinkowych,
        // żeby dzielenie było dokładne
        double a = (pbY - paY) / d;
        double b = paY - a * paX;
        x = (point.y - b) / a;
    }

    // Jeżeli x < point.x
    if (x < point.x) {
        counter++;
    }
}

// Zwróć prawdę, jeżeli counter jest nieparzysty, a fałsz w
// przeciwnym przypadku
return counter % 2 != 0;
}
}

```

junit testy:

Krok 4.

Dodaj do projektu bibliotekę JUnit,. Napisz trzy testy. W każdym stwórz wielokąt oraz punkt - tak aby w kolejnych testach:

- punkt leżał w wielokącie,
- punkt leżał pod wielokątem,
- punkt leżał na prawo od wielokąta.

Uwaga! W przypadku tego oraz pozostałych kroków wymagających testowania, w razie nieumiejętności wykorzystania biblioteki JUnit, część punktów można zdobyć pisząc metody klasy Main, wykonujące działania, które mają zostać przetestowane.

Wyobraźmy sobie dwuwymiarową mapę, na której położenie jest opisane za pomocą współrzędnych x i y , gdzie punkt $(0, 0)$ znajduje się w lewej górnej krawędzi mapy. Mapa jest uproszczonym modelem, nie uwzględniającym krzywizny Ziemi, który może być wykorzystany np. na potrzeby gry komputerowej. Na mapie znajdują się lądy opisane za pomocą wielokątów.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import java.util.Arrays;

public class PolygonTest {

    // Pomocnicza metoda, która buduje nasz testowy wielokąt (kwadrat)
    // Zaczyna się w (10, 10) i kończy w (30, 30)
    private Polygon createTestPolygon() {
        return new Polygon(Arrays.asList(
            new Point(10, 10), // Lewy górny
            new Point(30, 10), // Prawy górny
            new Point(30, 30), // Prawy dolny
            new Point(10, 30)  // Lewy dolny
        ));
    }

    // TEST 1: Punkt w wielokącie
```

```

@Test
public void testPointInsidePolygon() {
    Polygon polygon = createTestPolygon();

    // Punkt (20, 20) leży idealnie w środku naszego kwadratu
    Point p = new Point(20, 20);

    // Oczekujemy, że metoda inside() zwróci true.
    // Drugi argument to opcjonalna wiadomość w przypadku błędu.
    assertTrue(polygon.inside(p), "Punkt leżący w środku powinien
zwracać true.");
}

// TEST 2: Punkt pod wielokątem
@Test
public void testPointBelowPolygon() {
    Polygon polygon = createTestPolygon();

    // Skoro Y rośnie w dół, a nasz kwadrat kończy się na Y = 30,
    // to punkt z Y = 40 będzie pod nim.
    Point p = new Point(20, 40);

    // Oczekujemy, że metoda inside() zwróci false.
    assertFalse(polygon.inside(p), "Punkt pod wielokątem nie leży
wewnątrz (powinno być false).");
}

// TEST 3: Punkt na prawo od wielokąta
@Test
public void testPointRightOfPolygon() {
    Polygon polygon = createTestPolygon();

    // X rośnie w prawo. Kwadrat kończy się na X = 30,
    // więc punkt z X = 40 będzie na prawo od niego.
    Point p = new Point(40, 20);

    // Oczekujemy, że metoda inside() zwróci false.
    assertFalse(polygon.inside(p), "Punkt na prawo od wielokąta
nie leży wewnątrz (powinno być false).");
}

```

```
}  
}
```

Napisz klasę City dziedziczącą po Polygon. W klasie zdefiniuj publiczną ostateczną zmienną center - punkt będący środkiem miasta oraz prywatną nazwę miasta. Napisz publiczny konstruktor, takie dwa argumenty oraz liczbę zmiennoprzecinkową - długość ściany muru. Konstruktor powinien zapisać w polu klasy środek miasta oraz nazwę, a w liście punktów odziedziczonej z klasy Polygon, wierzchołki kwadratu stanowiącego mury.

```
import java.util.Arrays;  
  
public class City extends Polygon {  
    // Publiczna ostateczna zmienna (final) - środek miasta  
    public final Point center;  
  
    // Prywatna zmienna - nazwa miasta  
    private String name;  
  
    // Publiczny konstruktor z trzema argumentami  
    public City(Point center, String name, double wallLength) {  
  
        // Wywołujemy konstruktor klasy Polygon (super), przekazując  
        // listę 4 wierzchołków.  
        // Połowa długości muru  
        // Zakładamy, że klasa Point przyjmuje współrzędne x i y (np.  
        // typu double).  
        super(Arrays.asList(  
            // Lewy górny róg (x - połowa, y - połowa)  
            new Point(center.x - (wallLength / 2.0), center.y -  
            (wallLength / 2.0)),  
  
            // Prawy górny róg (x + połowa, y - połowa)  
            new Point(center.x + (wallLength / 2.0), center.y -  
            (wallLength / 2.0)),  
  
            // Prawy dolny róg (x + połowa, y + połowa)  
            new Point(center.x + (wallLength / 2.0), center.y +  
            (wallLength / 2.0)),  
        ));  
        this.center = center;  
        this.name = name;  
        this.wallLength = wallLength;  
    }  
}
```

```

        // Lewy dolny róg (x - połowa, y + połowa)
        new Point(center.x - (wallLength / 2.0), center.y +
(wallLength / 2.0))
    ));

    // Po wywołaniu super() możemy przypisać wartości do pól
naszej klasy
    this.center = center;
    this.name = name;
}
}

```

W klasie Land stwórz prywatną listę miast. Napisz metodę addCity(City), która doda miasto do tej listy. Miasto może zostać dodane wyłącznie jeżeli jego środek znajduje się na lądzie. W przeciwnym razie należy rzucić wyjątek RuntimeException, którego metoda getMessage() powinna wyświetlić nazwę miasta.

```

// KROK 7: Prywatna lista miast
// Od razu ją inicjalizujemy, aby uniknąć błędu
NullPointerException
private List<City> cities = new ArrayList<>();

// Konstruktor z Kroku 5
public Land(List<Point> points) {
    super(points);
}

// KROK 7: Metoda dodająca miasto
public void addCity(City city) {
    // Korzystamy z odziedziczonej metody inside(), przekazując
publiczny środek miasta.
    // "this.inside" oznacza, że sprawdzamy wielokąt (Land), na
rzecz którego wywołano metodę.
    if (this.inside(city.center)) {
        cities.add(city);
    } else {
        // Jeśli środek miasta leży poza lądem (metoda inside
zwróciła false),

```

```

        // rzucamy niewymagający łapania wyjątek RuntimeException.
        // Używamy gettera getName() (upewnij się, że masz go w
klasie City!),
        // aby przekazać nazwę miasta jako treść błędu.
        throw new RuntimeException(city.getName());
    }
}

// Opcjonalnie: getter dla listy miast, jeśli będzie potrzebna w
dalszych krokach
public List<City> getCities() {
    return cities;
}
}

```

W klasie Land stwórz prywatną listę miast. Napisz metodę addCity(City), która doda miasto do tej listy. Miasto może zostać dodane wyłącznie jeżeli jego środek znajduje się na lądzie. W przeciwnym razie należy rzucić wyjątek RuntimeException, którego metoda getMessage() powinna wyświetlić nazwę miasta.

```

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import java.util.Arrays;

public class LandTest {

    @Test
    public void testAddCityThrowsExceptionForCityInWater() {
        // 1. Tworzymy przykładowy ląd (np. kwadrat od 0,0 do 100,100)
        Land land = new Land(Arrays.asList(
            new Point(0, 0),
            new Point(100, 0),
            new Point(100, 100),
            new Point(0, 100)
        ));

        // 2. Tworzymy miasto, którego środek ewidentnie leży POZA
naszym lądem
        // Środek w punkcie (200, 200) jest daleko poza kwadratem

```

```

kończącym się na 100
    String testCityName = "Atlantyda";
    City waterCity = new City(new Point(200, 200), testCityName,
10.0);

    // 3. Testujemy rzucanie wyjątku
    // assertThrows przyjmuje typ oczekiwanego wyjątku oraz
wyrażenie lambda () -> z kodem, który ma go wywołać
    RuntimeException exception =
assertThrows(RuntimeException.class, () -> {
        land.addCity(waterCity);
    }, "Metoda addCity powinna rzucić RuntimeException, gdy miasto
jest poza lądem.");

    // 4. Testujemy poprawność nazwy wyświetlanej przez
getMessage()
    assertEquals(testCityName, exception.getMessage(), "Wiadomość
błędu powinna być równa nazwie miasta.");
}
}

```

W klasie City dodaj prywatne pole logiczne port określające, czy miasto jest portowe. Miasto jest miastem portowym, jeżeli co najmniej jeden z wierzchołków kwadratu stanowiącego mury miasta znajduje się poza lądem. Pole to powinna ustawiać metoda znająca zarówno obiekt City, jak i obiekt Land, na którym istnieje lub do którego dodawane jest miasto. Samodzielnie wybierz odpowiednie położenie definicji i wywołania oraz symbol tej metody.

```

// Dodaj to w klasie Polygon:
public List<Point> getPoints() {
    return points;
}

```

// Zmiany w klasie City:

```

public class City extends Polygon {
    public final Point center;
}

```

```
private String name;
```

```
// NOWE POLE: Flaga określająca, czy miasto jest portem
```

```
private boolean port;
```

```
public City(Point center, String name, double wallLength) {
```

```
    // ... (konstruktor pozostaje bez zmian, taki jak pisaliśmy wcześniej)
```

```
}
```

```
// NOWE METODY: Setter i Getter dla pola port
```

```
public void setPort(boolean port) {
```

```
    this.port = port;
```

```
}
```

```
public boolean isPort() {
```

```
    return port;
```

```
}
```

```
public String getName() {
```

```
    return name;
```

```
}
```

```
}
```

```
// Zmodyfikowana metoda w klasie Land:
```

```
public void addCity(City city) {
```

```
    // 1. Sprawdzamy czy środek leży na lądzie (tak jak w poprzednim kroku)
```

```
    if (this.inside(city.center)) {
```

```
        // 2. LOGIKA MIASTA PORTOWEGO
```

```
        boolean isPortCity = false;
```

```
        // Pobieramy wierzchołki murów miasta (które dziedziczą się z Polygon)
```

```
        // i sprawdzamy każdy z nich.
```

```
        for (Point vertex : city.getPoints()) {
```

```
            // Jeśli chociaż jeden wierzchołek leży POZA lądem (!this.inside)
```

```
            if (!this.inside(vertex)) {
```

```

        isPortCity = true;
        break; // Znaleźliśmy punkt w wodzie, nie musimy sprawdzać reszty
    }
}

// 3. Ustawiamy status w obiekcie City
city.setPort(isPortCity);

// 4. Dodajemy miasto do listy lądu
cities.add(city);

} else {
    throw new RuntimeException(city.getName());
}
}

```

Zdefiniuj klasę Resource. W klasie Resource zdefiniuj publiczny typ wyliczeniowy Type {Coal, Wood, Fish}. Klasa Resource powinna posiadać dwa publiczne, ostateczne pola: punkt (Point) określający rozmieszczenie pozycję zasobu na mapie oraz typ (Type) zasobu. Napisz konstruktor ustawiający te pola.

```
public class Resource {
```

```

    // KROK X: Definicja publicznego typu wyliczeniowego
    // Wewnątrz klasy możemy definiować inne typy (podobnie jak
robiiliśmy to z klasą statyczną).
    // Enum to specjalna klasa, która z góry definiuje listę
dozwolonych wartości.
    public enum Type {
        Coal,
        Wood,
        Fish
    }

    // KROK X: Publiczne, ostateczne (final) pola
    // Skoro pole jest 'final', musi zostać przypisane w konstruktorze
i nigdy więcej nie zmieni swojej wartości.
    // Dzięki temu, że pola są publiczne i finalne jednocześnie, nie
musimy pisać dla nich getterów (np. getPosition()),

```

```

// bo nikt i tak nie będzie w stanie ich przypadkowo nadpisać.
public final Point position;
public final Type type;

// KROK X: Konstruktor ustawiający te pola
public Resource(Point position, Type type) {
    this.position = position;
    this.type = type;
}
}

```

Załóżmy bez sprawdzenia, że węgiel i drewno znajdują się na lądzie, a ryby w wodzie. W klasie City utwórz zbiór (Set) obiektów Resource.Type o nazwie resources i dostępie pakietowym. W tej samej klasie napisz metodę addResourcesInRange, która przyjmie listę obiektów Resource oraz liczbę zmiennoprzecinkową range i umieści w zbiorze typy tych zasobów, które znajdują się w odległości nie większej niż range od środka miasta. Ryby powinny być uwzględniane wyłącznie w miastach portowych.

```

import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class City extends Polygon {
    public final Point center;
    private String name;
    private boolean port;

    // KROK X: Zbiór zasobów o dostępie pakietowym (brak słowa
private/public!)
    // Inicjalizujemy go jako HashSet, czyli najpopularniejszą
implementację Set.
    Set<Resource.Type> resources = new HashSet<>();

    public City(Point center, String name, double wallLength) {
        // ... (Twój poprzedni kod konstruktora)
    }

    // KROK X: Metoda dodająca zasoby w zasięgu

```

```

    public void addResourcesInRange(List<Resource> resourceList,
double range) {

    // Iterujemy po wszystkich zasobach z przekazanej listy
    for (Resource res : resourceList) {

        // 1. Obliczanie odległości (wzór na odległość dwóch
punktów - Pitagoras)
        double dx = this.center.x - res.position.x;
        double dy = this.center.y - res.position.y;
        double distance = Math.sqrt((dx * dx) + (dy * dy));

        // 2. Sprawdzamy, czy zasób mieści się w zadanym promieniu
(range)
        if (distance <= range) {

            // 3. Logika dodawania zasobu
            if (res.type == Resource.Type.Fish) {
                // Ryby dodajemy tylko wtedy, gdy miasto jest
portem

                if (this.port) {
                    this.resources.add(res.type);
                }
            } else {
                // Węgiel i drewno dodajemy normalnie (zgodnie z
założeniem w poleceniu)
                this.resources.add(res.type);
            }
        }
    }

    // ... (pozostałe metody, getterry i setterry z poprzednich kroków)
}

```

Napisz klasę CityTest zawierającą sparametryzowany test zawierający:

- poprawne dodanie węgla,
- nieudaną próbę dodania drewna spoza zasięgu miasta,
- poprawne dodanie ryb do miasta portowego,
- nieudaną próbę dodania ryb do miasta nieportowego.

Test będzie wymagał definicji miasta śródlądowego i morskiego, co można wykonać na wiele sposobów, np. przez ustawienie ich jako pola klasy.

Test powinien przyjmować argumenty: miasto, zasób, oczekiwana wartość logiczna.

Sposób parametryzacji testu jest dowolny. Test, przy użyciu pakietowego dostępu do pola `City.resources` powinien sprawdzać, czy testowany zasób został dodany do zbioru.

Dany jest plik `map.svg` zawierający przykładową mapę. Mapa zawiera niebieski prostokąt reprezentujący wodę, który można pominąć w dalszych rozważaniach oraz za pomocą znaczników:

- polygon o kolorze zielonym - ląd,
- rect o kolorze czerwonym - miasto,
- circle o kolorze czarnym, brązowym, jasnoniebieskim - zasoby, odpowiednio węgiel, drewno, ryby.
- text - nazwy miast.

Symbole zasobów nie będą rozważane w dalszej części zadania.

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.Arguments;
import org.junit.jupiter.params.provider.MethodSource;

import java.util.Arrays;
import java.util.stream.Stream;

import static org.junit.jupiter.api.Assertions.assertEquals;

public class CityTest {

    // =====
    // ŹRÓDŁO DANYCH DLA TESTU (MethodSource)
    // =====
    // Ta statyczna metoda przygotowuje i zwraca "paczkę" argumentów
    // dla naszego testu.
    // Każdy wpis Arguments.of(...) to jeden osobny przypadek testowy.
    static Stream<Arguments> provideResourcesForCity() {

        // 1. Definiujemy miasto śródlądowe (nieportowe)
```

```
City inlandCity = new City(new Point(10, 10), "Warszawa",
10.0);
inlandCity.setPort(false); // Jawnie oznaczamy jako lądowe

// 2. Definiujemy miasto morskie (portowe)
City portCity = new City(new Point(100, 100), "Gdańsk", 10.0);
portCity.setPort(true); // Oznaczamy jako port

return Stream.of(
    // Argumenty w kolejności: [Miasto], [Zasób],
[Oczekiwany wynik logiczny]

    // PRZYPADEK 1: Poprawne dodanie węgla (w zasięgu)
    // Węgiel leży w tym samym punkcie co miasto (10, 10),
odległość = 0
    Arguments.of(inlandCity, new Resource(new Point(10,
10), Resource.Type.Coal), true),

    // PRZYPADEK 2: Nieudana próba dodania drewna spoza
zasięgu
    // Drewno leży w punkcie (50, 50), odległość od miasta
to znacznie więcej niż nasz testowy zasięg (np. 10.0)
    Arguments.of(inlandCity, new Resource(new Point(50,
50), Resource.Type.Wood), false),

    // PRZYPADEK 3: Poprawne dodanie ryb do miasta
portowego (w zasięgu)
    // Ryby leżą w punkcie (100, 100), idealnie w centrum
miasta morskiego
    Arguments.of(portCity, new Resource(new Point(100,
100), Resource.Type.Fish), true),

    // PRZYPADEK 4: Nieudana próba dodania ryb do miasta
nieportowego (w zasięgu)
    // Ryby leżą blisko miasta śródlądowego (10, 10), ale
zgodnie z logiką nie powinny zostać dodane
    Arguments.of(inlandCity, new Resource(new Point(10,
10), Resource.Type.Fish), false)
);
```

```

    }

    // =====
    // WŁAŚCIWY TEST SPARAMETRYZOWANY
    // =====
    // Adnotacja informująca, że to test sparametryzowany, a jego
    // danymi będzie metoda u góry.
    @ParameterizedTest
    @MethodSource("provideResourcesForCity")
    public void testAddResourcesInRange(City testCity, Resource
testResource, boolean expectedResult) {

        // Zasięg, którego używamy w naszych obliczeniach (odpowiada
        // za odległość z twierdzenia Pitagorasa)
        double testRange = 10.0;

        // Czyścimy zbiór zasobów miasta na wypadek, gdyby ten sam
        // obiekt miasta był używany w kolejnym teście
        testCity.resources.clear();

        // Wywołujemy naszą metodę z poprzedniego kroku.
        // Oczekuje ona Listy, więc pakujemy nasz pojedynczy
        // testResource za pomocą Arrays.asList
        testCity.addResourcesInRange(Arrays.asList(testResource),
testRange);

        // KROK X: Weryfikacja
        // Sprawdzamy, czy w pakietowym zbiorze "resources" faktycznie
        // pojawił się typ tego zasobu.
        boolean actuallyAdded =
testCity.resources.contains(testResource.type);

        // Asersja (porównanie) wyniku oczekiwanego z tym, co
        // faktycznie zadziało się w kodzie
        assertEquals(expectedResult, actuallyAdded,
            "Test dla zasobu " + testResource.type + " z
oczekiwanym statusem " + expectedResult + " zakończył się
niepowodzeniem.");
    }

```

```
}
```

Zapoznaj się z dołączoną klasą MapParser służącą do parsowania tego pliku. Klasa wykorzystuje bibliotekę Jackson Dataformat XML. Dołącz ją do projektu. W klasie MapParser znajduje się lista obiektów typu Label zawierającego reprezentację znaczników <text>. Metody parse i parseText prezentują proces konwersji zawartości znacznika na obiekt. W klasie MapParser stwórz analogiczne do istniejącej listy nazw, nowe listy łądów i miast i wypełnij je wzorując się na istniejącym kodzie. Napisz publiczny akcesor do listy łądów.

```
import
com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlElementWrap
per;
import
com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlProperty;
import java.util.ArrayList;
import java.util.List;

public class MapParser {
    // Istniejąca lista Label (znaczniki <text>)
    @JacksonXmlProperty(localName = "text")
    @JacksonXmlElementWrapper(useWrapping = false)
    private List<Label> labels;

    // Nowe listy łądów i miast
    private List<Land> lands = new ArrayList<>();
    private List<City> cities = new ArrayList<>();

    // Klasy pomocnicze do mapowania XML (Jackson)
    private static class RectData {
        public double x, y, width, height;
    }

    private static class PolygonData {
        public String points;
    }
}
```

```

// Akcesor do listy lądów
public List<Land> getLands() {
    return lands;
}

// Metoda pomocnicza parsująca miasto z rect
public void parseCity(RectData data) {
    // Obliczamy środek miasta na podstawie lewego górnego rogu i
wymiarów
    double centerX = data.x + (data.width / 2.0);
    double centerY = data.y + (data.height / 2.0);

    // Nazwa tymczasowo jako null (zostanie dopasowana później z
Label)
    City city = new City(new Point(centerX, centerY), null,
data.width);
    cities.add(city);
}

// Metoda parsująca ląd z polygon
public void parseLand(PolygonData data) {
    String[] pairs = data.points.split("\n");
    List<Point> points = new ArrayList<>();
    for (String p : pairs) {
        String[] coords = p.split(",");
        points.add(new Point(Double.parseDouble(coords[0]),
Double.parseDouble(coords[1])));
    }
    lands.add(new Land(points));
}
}

```